Protocolos de Comunicaciones.

Trabajo Práctico N°2

FCEFYN - UNC

2026



Alumno:

Cabero, Mauro Ezequiel.

Docentes:

Finochietto, Jorge.

Sposetti, Facundo.

Índice

1. Introducción **1**

2. Objetivos **1**

3. Desarrollo de los Módulos **1**
3.1. Generador LFSR Galois (`lfsr.v`) 1
3.2. Verificador del Generador (`checker_lfsr.v`) 2
3.3. Entorno de Verificación (*Testbench*) 5
3.3.1. Resultados de Simulación en Vivado Simulator (XSim) 6

4. Recursos y Utilización de Área (Celdas Lógicas) **6**
4.1. Desglose Jerárquico por Módulo 7

5. Análisis de Consumo de Potencia **7**

6. Bibliografía y Referencias **8**

1. Introducción

Los Registros de Desplazamiento con Realimentación Lineal (*Linear Feedback Shift Register* - LFSR) constituyen una estructura fundamental en sistemas de telecomunicaciones, criptografía y procesamiento digital de señales. Se emplean ampliamente como generadores de secuencias pseudoaleatorias (*PRBS*) para sincronización de tramas, codificación de canal y pruebas de tasa de error de bit (*BERT*).

En este trabajo se implementa y valida un sistema completo compuesto por un generador LFSR basado en topología Galois de 16 bits y un módulo receptor/verificador (*Checker*) paralelo con capacidad de auto-sembrado e histéresis frente a ráfagas de error.

2. Objetivos

- Diseñar un generador de secuencias pseudoaleatorias LFSR de 16 bits en arquitectura Galois con prevención de estado de bloqueo nulo (*zero-lock*).
- Implementar un módulo *Checker* paralelo que valide la secuencia sin conocimiento previo de la semilla inicial mediante auto-sembrado en estado `UNLOCKED`.
- Incorporar una máquina de estados finitos (FSM) con un pequeño umbral de desincronización para tolerar fallos transitorios antes de declarar pérdida de sincronismo.
- Desarrollar un banco de pruebas (*Testbench*) modular con aserciones inmediatas (*immediate assertions*) e inyección de fallos mediante compuertas XOR combinacionales.

3. Desarrollo de los Módulos

3.1. Generador LFSR Galois (`lfsr.v`)

El generador implementa el polinomio sobre el bus de 16 bits actualizando su estado interno en cada flanco positivo de reloj cuando la señal `i_valid` está activa. Soporta reset asincrónico (`i_rst`) y reinicio sincrónico (`i_soft_rst`) para carga dinámica de semilla.

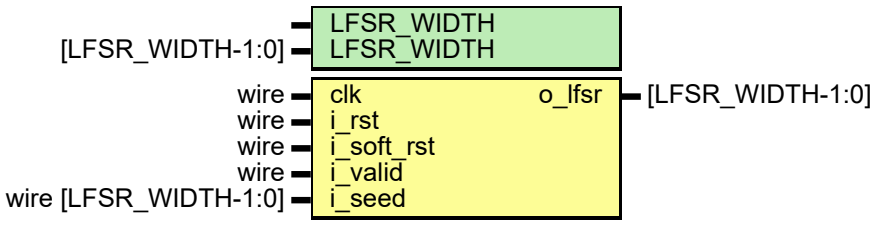


Figura 1: Diagrama esquemático y flujo lógico del generador Galois LFSR

```
1 `timescale 1ns / 100ps
2
3 module lfsr #(
4     parameter LFSR_WIDTH = 16,
5     parameter [LFSR_WIDTH-1:0] LFSR_SEED = 16'd65535
6 )(
```

```
7     input  wire                  clk,
8     input  wire                  i_rst,
9     input  wire                  i_soft_rst,
10    input  wire                  i_valid,
11    input  wire [LFSR_WIDTH-1:0] i_seed,
12    output reg  [LFSR_WIDTH-1:0] o_lfsr
13);
14    wire internal_feedback;
15
16    always @(posedge clk or posedge i_rst) begin
17        if (i_rst)
18            o_lfsr <= LFSR_SEED;
19        else if (i_soft_rst)
20            o_lfsr <= i_seed;
21        else if (i_valid) begin
22            o_lfsr[0]    <= internal_feedback;
23            o_lfsr[1]    <= o_lfsr[0];
24            o_lfsr[2]    <= o_lfsr[1] ^ internal_feedback;
25            o_lfsr[3]    <= o_lfsr[2] ^ internal_feedback;
26            o_lfsr[4]    <= o_lfsr[3];
27            o_lfsr[5]    <= o_lfsr[4] ^ internal_feedback;
28            o_lfsr[15:6] <= o_lfsr[14:5];
29        end
30        else
31            o_lfsr <= o_lfsr;
32    end
33
34    assign internal_feedback = o_lfsr[15] ^ (o_lfsr == 16'd0);
35
36endmodule
```

Listing 1: Código RTL del Generador LFSR

3.2. Verificador del Generador (`checker_lfsr.v`)

El verificador utiliza una arquitectura de dos bloques (secuencial y combinacional) para gobernar la FSM. En estado `UNLOCKED`, captura el bus de entrada `i_output_lfsr` para predecir el siguiente estado esperado. Al acumular aciertos consecutivos transiciona a `LOCKED`; ante fallos consecutivos supera el umbral `COUNTER_FAIL` antes de transicionar nuevamente a `UNLOCKED`.

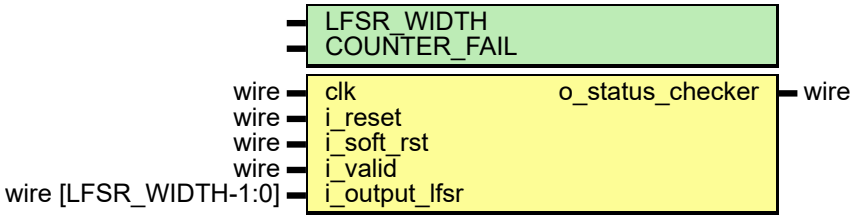


Figura 2: Diagrama esquemático y flujo lógico del verificador (*Checker*).

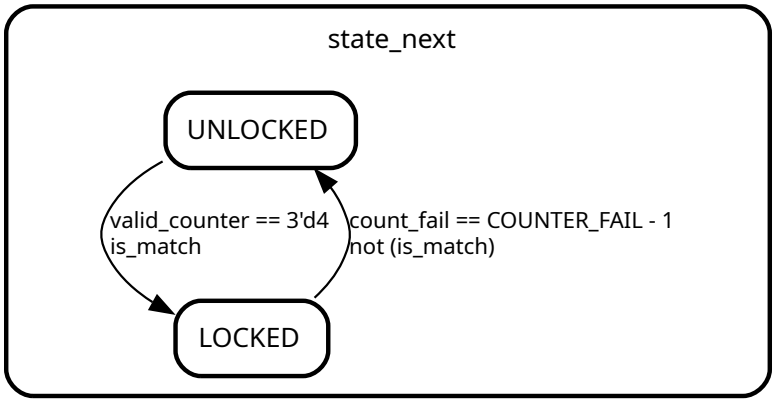


Figura 3: Diagrama de transiciones de la Máquina de Estados Finitos (FSM) del verificador.

```
1  `timescale 1ns / 100ps
2
3  module checker_lfsr #(
4      parameter LFSR_WIDTH   = 16,
5      parameter COUNTER_FAIL = 3
6  )(
7      input  wire                  clk,
8      input  wire                  i_reset,
9      input  wire                  i_soft_rst,
10     input  wire                  i_valid,
11     input  wire [LFSR_WIDTH-1:0] i_output_lfsr,
12     output wire                  o_status_checker
13 );
14     localparam UNLOCKED = 1'b0;
15     localparam LOCKED   = 1'b1;
16
17     reg        state, state_next;
18     reg [2:0]  valid_counter, valid_counter_next;
19     reg [1:0]  count_fail, count_fail_next;
20     reg [31:0] count_bits_ok, count_bits_ok_next;
21     reg [LFSR_WIDTH-1:0] internal_lfsr, internal_lfsr_next;
22
23     wire feedback;
24     wire [LFSR_WIDTH-1:0] expected_next_lfsr;
25     wire is_match;
26
27     assign feedback = internal_lfsr[15] ^ !(internal_lfsr == 16'd0);
28
29     assign expected_next_lfsr[0]    = feedback;
30     assign expected_next_lfsr[1]    = internal_lfsr[0];
31     assign expected_next_lfsr[2]    = internal_lfsr[1] ^ feedback;
32     assign expected_next_lfsr[3]    = internal_lfsr[2] ^ feedback;
33     assign expected_next_lfsr[4]    = internal_lfsr[3];
34     assign expected_next_lfsr[5]    = internal_lfsr[4] ^ feedback;
35     assign expected_next_lfsr[15:6] = internal_lfsr[14:5];
36
37     assign is_match = (expected_next_lfsr == i_output_lfsr);
38     assign o_status_checker = state;
39
40     always @(posedge clk or posedge i_reset) begin
41         if (i_reset) begin
```

```
42              state          <= UNLOCKED;
43              valid_counter  <= 3'd0;
44              count_bits_ok  <= 32'd0;
45              count_fail     <= 2'd0;
46              internal_lfsr  <= {LFSR_WIDTH{1'b0}};
47          end
48          else if (i_soft_rst) begin
49              state          <= UNLOCKED;
50              valid_counter  <= 3'd0;
51              count_bits_ok  <= 32'd0;
52              count_fail     <= 2'd0;
53              internal_lfsr  <= {LFSR_WIDTH{1'b0}};
54          end
55          else if (i_valid) begin
56              state          <= state_next;
57              valid_counter  <= valid_counter_next;
58              count_bits_ok  <= count_bits_ok_next;
59              count_fail     <= count_fail_next;
60              internal_lfsr  <= internal_lfsr_next;
61          end
62      end
63
64      always @(*) begin
65          state_next          = state;
66          valid_counter_next  = valid_counter;
67          count_bits_ok_next  = count_bits_ok;
68          count_fail_next     = count_fail;
69          internal_lfsr_next  = internal_lfsr;
70
71          case (state)
72              UNLOCKED: begin
73                  internal_lfsr_next = i_output_lfsr;
74                  if (is_match) begin
75                      count_bits_ok_next = count_bits_ok + 32'd1;
76                      if (valid_counter >= 3'd3) begin
77                          state_next = LOCKED;
78                          valid_counter_next = 3'd0;
79                      end else begin
80                          valid_counter_next = valid_counter + 3'd1;
81                      end
82                  end else begin
83                      valid_counter_next = 3'd0;
84                  end
85              end
86
87              LOCKED: begin
88                  internal_lfsr_next = expected_next_lfsr;
89                  if (is_match) begin
90                      count_bits_ok_next = count_bits_ok + 32'd1;
91                      count_fail_next    = 2'd0;
92                  end else begin
93                      count_fail_next = count_fail + 2'd1;
94                      if (count_fail == COUNTER_FAIL - 1) begin
95                          valid_counter_next = 3'd0;
96                          count_bits_ok_next = 32'd0;
97                          count_fail_next    = 2'd0;
98                          state_next         = UNLOCKED;
99                      end
100                 end
101             end
102
```

```
103             default: begin
104                 state_next          = UNLOCKED;
105                 valid_counter_next  = 3'd0;
106                 count_bits_ok_next  = 32'd0;
107                 count_fail_next     = 2'd0;
108             end
109         endcase
110     end
111
112 endmodule
```

Listing 2: Código RTL del Checker Paralelo

3.3. Entorno de Verificación (*Testbench*)

Se implementó una infraestructura de verificación en SystemVerilog (`tb_lfsr_system.sv`) configurada para operar a una frecuencia base de 100 MHz ($T_{\text{clk}} = 10\,\text{ns}$).

El entorno incorpora un mecanismo de inyección de fallas por emulación de canal mediante una compuerta (XOR) combinacional sobre el bus paralelo de datos.

- **Prueba 1 – Sincronización Nominal** (`test_normal`):
  Evalúa la convergencia hacia el estado `LOCKED` en régimen libre de ruido. Tras la fase de auto-sembrado, la aserción verifica formalmente la captura de sincronismo (checker_status = 1) al cabo de $N \geq 5$ ciclos válidos consecutivos.
- **Prueba 2 – Transición Forzada a Desenganche** (`test_forzar_error`):
  Inyecta una máscara de perturbación destructiva total ($\mathbf{M}_{\text{error}}$ = `16'hFFFF`) durante 3 periodos consecutivos de reloj. La aserción valida la superación del umbral `COUNTER_FAIL` y la correcta transición a `UNLOCKED` (checker_status = 0).
- **Prueba 3 – Reinicio con Semilla Estocástica** (`test_seed_random`):
  Aplica la señal `i_soft_rst` con un vector de inicialización pseudoaleatorio generado por la primitiva `$random`, comprobando la capacidad del verificador de readaptarse y converger al nuevo régimen sin pérdida de coherencia.
- **Prueba 4 – Análisis de Frontera en Umbral** (`test_frontera_4v_1i`):
  Introduce una alteración mono-bit en el cuarto ciclo consecutivo de validación, interrumpiendo la secuencia previa al enganche. La aserción constata la reinicialización del contador interno y la permanencia en `UNLOCKED`.
- **Prueba 5 – Inmunidad frente a Reset Espurio** (`test_reset_aleatorio`):
  Dispara un evento de reinicio de hardware (`reset`) en un instante temporal aleatorio dentro del estado `LOCKED`, verificando la posterior re-adquisición automática de sincronía tras cesar la condición de reset.
- **Prueba 6 – Histéresis y Margen de Tolerancia** (`test_histeresis_tolerancia`):
  En estado estacionario `LOCKED`, somete al receptor a una ráfaga de 2 fallos consecutivos (máximo admisible para COUNTER_FAIL = 3) seguida de una muestra válida. La aserción ratifica la permanencia en `LOCKED` y el blanqueo del contador de fallas.
- **Prueba 7 – Re-alimentación de Error como Semilla** (`test_error_como_semilla`):
  Introduce una perturbación ($\mathbf{M}_{\text{error}}$ = `16'h1010`), captura el vector resultante en el canal y lo transfiere como nueva semilla mediante `i_soft_rst`. Se verifica por aserción la re-sincronización exitosa bajo las nuevas condiciones de régimen.

3.3.1. Resultados de Simulación en Vivado Simulator (XSim)

La totalidad de los vectores de prueba fueron compilados y ejecutados satisfactoriamente en el entorno Vivado Simulator v2024.2, registrando cumplimiento estricto de todas las aserciones formales sin advertencias ni violaciones funcionales. A continuación se reproduce la transcripción de salida del registro de eventos para los escenarios críticos evaluados:

```
=======================================
  Test seleccionado: 1
=======================================
[0] Aplicando Reset Sincronizado...
[45000] TEST 1: Sincronizacion Normal
   -> [EXITO] Assertion Pass: El Checker alcanzo el estado LOCKED.

=======================================
  Test seleccionado: 6
=======================================
[0] Aplicando Reset Sincronizado...
[45000] TEST 6: Tolerancia a fallos (2 errores, 1 acierto)
[65000] Inyectando 2 errores consecutivos (Tolerancia maxima)...
[85000] Enviando dato correcto (deberia limpiar los fallos)...
   -> [EXITO] Assertion Pass: El Checker aplico histeresis y mantuvo el
    candado.

=======================================
  Test seleccionado: 7
=======================================
[0] Aplicando Reset Sincronizado...
[45000] TEST 7: Inyectar error y usarlo como nueva semilla (Soft Reset)
[65000] Bus corrompido capturado: efe0
[65000] Aplicando Soft Reset usando el dato corrupto como semilla...
   -> [EXITO] Assertion Pass: El sistema adopto la nueva semilla y se
    resincronizo.
```

Listing 3: Registro de ejecución y verificación formal por aserciones (Vivado Tcl Console)

4. Recursos y Utilización de Área (Celdas Lógicas)

A partir del reporte de implementación en Vivado sobre el módulo integrador (`top_system_lfsr`), se obtuvo la utilización global de recursos.

Cuadro 1: Utilización global de recursos en FPGA para el sistema integrado

Recurso	**Utilizados**	**Disponibles**	**% Utilización**
Slice LUTs (Compuertas lógicas)	49	20800	0.24 %
Slice Registers (Flip-Flops)	38	41600	0.09 %
Slices	16	8150	0.20 %
LUT as Logic	49	20800	0.24 %
Bonded IOB (Pines de E/S)	21	210	10.00 %
BUFGCTRL (Buffers de reloj)	1	32	3.13 %

4.1. Desglose Jerárquico por Módulo

La Tabla 2 detalla la distribución de compuertas y registros entre el generador y el verificador:

Cuadro 2: Distribución jerárquica de recursos (*Utilization by Hierarchy*)

Instancia / Módulo	**Slice LUTs**	**Slice Registers**	**Slices**	**LUT as Logic**
`top_system_lfsr` (Total)	**49**	**38**	**16**	**49**
→ `u_checker` (`checker_lfsr`)	34	22	12	34
→ `u_lfsr` (`lfsr`)	15	16	10	15

El módulo `u_checker` concentra el 69.4 % de la lógica combinacional (34 LUTs) y el 57.9 % de los registros (22 Flip-Flops) debido al predictor combinacional y a los contadores de la máquina de estados . Por su parte, el generador `u_lfsr` demanda únicamente 15 LUTs y 16 registros para el registro de desplazamiento Galois de 16 bits.

5. Análisis de Consumo de Potencia

A partir del reporte de consumo generado tras la implementación en Vivado (*Power Analysis*), se evaluó la disipación energética total en la FPGA Arty A7-35T. El diseño presenta un consumo sumamente acotado, dominado en un 98 % por la disipación estática propia del silicio.

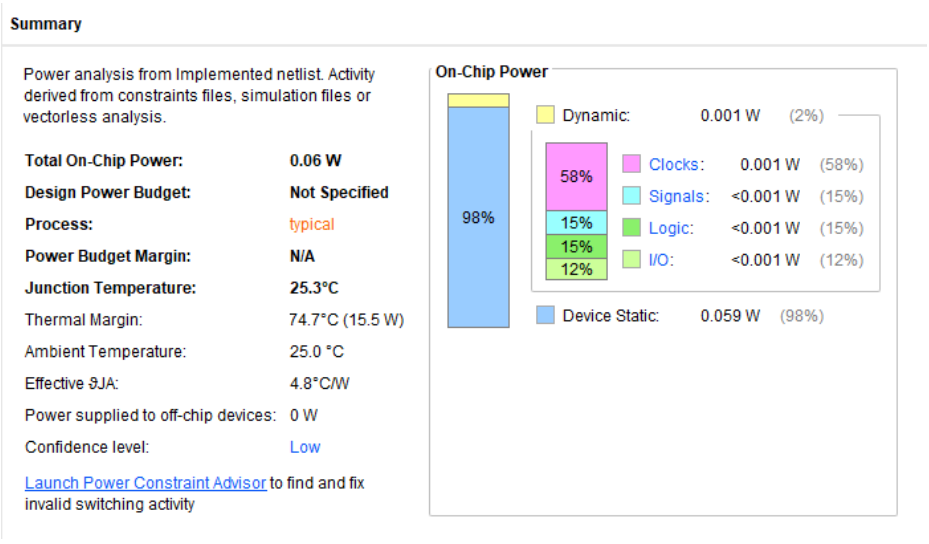


Figura 4: Reporte de disipación de potencia en el chip (*On-Chip Power*).

Cuadro 3: Resumen de potencia disipada en el chip

Componente de Potencia	**Valor** [W]	**Porcentaje** [%]
Potencia Dinámica (*Dynamic*)	0.001	2
Potencia Estática del Dispositivo (*Device Static*)	0.059	98
Potencia Total en Chip (*Total On-Chip Power*)	0.060	100

Dentro de la potencia dinámica (0.001 W), la distribución porcentual de los recursos internos se compone de:

- **Árbol de Reloj (*Clocks*):** 58 % (0.001 W).
- **Enrutamiento de Señales (*Signals*):** 15 % ($<$ 0.001 W).
- **Lógica Interna (*Logic*):** 15 % ($<$ 0.001 W).
- **Pines de Entrada/Salida (*I/O*):** 12 % ($<$ 0.001 W).

6. Bibliografía y Referencias

1. **ChipVerify**. *Verilog Tutorial & Verification Guides*: `https://chipverify.com/tutorials/verilog`
2. **HDLBits**. *Verilog Digital Circuit Practice*: `https://hdlbits.01xz.net/wiki/Main_Page`